GitHub Trending Top 10

每日热门开源项目深度解读

2026-04-18

今日榜单

1. **EvoMap/evolver** JavaScript
   The GEP-Powered Self-Evolution Engine for AI Agents. Genome Evolution Protocol. ... 2天

2. **lsdefine/GenericAgent** Python
   Self-evolving agent: grows skill tree from 3.3K-line seed, achieving full system... 3天

3. **SimoneAvogadro/android-reverse-engineering-skill** Shell
   Claude Code skill to support Android app's reverse engineering 2天

4. **BasedHardware/omi** Dart
   AI that sees your screen, listens to your conversations and tells you what to do 2天

5. **Lordog/dive-into-llms** Jupyter Notebook
   《动手学大模型Dive into LLMs》系列编程实践教程 4天

6. **Donchitos/Claude-Code-Game-Studios** Shell
   Turn Claude Code into a full game dev studio — 49 AI agents, 72 workflow skills,... 2天

7. **jamiepine/voicebox** TypeScript
   The open-source voice synthesis studio 5天

8. **lukilabs/craft-agents-oss** TypeScript
   2天

9. **Tracer-Cloud/opensre** Python
   Build your own AI SRE agents. The open source toolkit for the AI era ✨ 2天

10. **obra/superpowers** Shell
    An agentic skills framework & software development methodology that works. 2天

#1 JavaScript 连续 2 天

evolver

The GEP-Powered Self-Evolution Engine for AI Agents. Genome Evolution Protocol. | evomap.ai

Stars **4,685** Forks **454** Today **+737**

https://github.com/EvoMap/evolver

项目简介：Evolver 是一个基于“基因组进化协议”（GEP）的 AI 智能体自我进化引擎。它旨在解决当前 AI 应用开发中，对智能体的调整往往依赖临时、零散的提示词修改（prompt tweaks）的问题，通过一套协议化的方法，将这些经验转化为可审计、可复用的“进化资产”。

核心功能：1. **协议化进化**：依据 GEP 协议，将进化过程标准化，生成结构化的进化提示。2. **资产复用**：提供“基因”（Gene）和“胶囊”（Capsule）等资产库，用于匹配和解决运行时发现的问题。3. **审计追踪**：完整记录每一次进化事件（EvolutionEvent），确保进化过程可追溯。4. **网络协作**：可连接 EvoMap 网络，共享和验证进化资产，参与协作进化。

适用场景：主要面向需要构建和维护复杂 AI 智能体的开发者或团队。典型场景包括：希望将智能体调试经验系统化、产品化的团队；需要确保智能体行为可审计、可解释的合规性场景；以及参与开放式协作、希望共享智能体进化成果的开发者社区。

技术亮点：1. **GEP 协议设计**：将生物进化概念（基因、突变）工程化为可执行的协议，是核心创新。2. **与 Git 深度集成**：利用 Git 进行版本管理、回滚和影响范围计算，将进化过程与代码管理无缝结合。3. **明确的边界**：严格区分“提示生成”与“代码执行”，自身不自动修改代码或执行命令，安全性设计清晰。

上手建议：对于使用者，建议从克隆仓库、运行 `node index.js` 开始，观察其如何分析本地日志并生成 GEP 提示，理解基本工作流。对于贡献者或深度用户，应仔细阅读项目 Wiki 以理解 GEP 协议细节，并关注其正从“完全开源”转向“源码可用”的许可变更及其背后的社区动态。

#2 Python 连续 3 天

GenericAgent

Self-evolving agent: grows skill tree from 3.3K-line seed, achieving full system control with 6x less token consumption

Stars **3,996** Forks **424** Today **+845**

https://github.com/lsdefine/GenericAgent

项目简介：GenericAgent 是一个极简、自进化的自主智能体框架。它通过仅约 3K 行的核心代码，让大语言模型获得对本地计算机（浏览器、终端、文件系统等）的系统级控制权，核心解决传统智能体框架依赖大量预编程技能、上下文冗长且成本高昂的问题。

核心功能：1. **自进化机制**：每次完成任务后，自动将执行路径固化为可复用的技能，形成专属技能树。2. **极简架构**：核心代码约3K行，代理主循环仅约100行，依赖少。3. **强系统控制**：通过9个原子工具直接操控真实浏览器、终端、ADB等。4. **高令牌效率**：上下文窗口小于30K，远低于同类框架，通过分层记忆精准调用知识。

适用场景：适合希望构建高度个性化、能自动学习并固化工作流的开发者或高级用户。典型场景包括自动化重复性桌面操作（如数据抓取、软件操作）、将自然语言指令转化为可重复执行的系统级脚本，或作为研究智能体自我扩展能力的实验平台。

技术亮点：其“技能结晶化”机制是核心创新，将动态的任务探索过程转化为静态、可高效调用的技能节点，实现了能力的指数级积累。此外，它通过注入真实浏览器来保持登录会话，以及采用分层记忆系统来保证高成功率与低成本，在工程实现上非常巧妙。

上手建议：建议从项目 README 的“快速开始”部分入手，按照标准安装步骤配置API密钥并运行示例，直观感受其工作流程。若想深入理解或贡献，可重点阅读其约100行的代理主循环代码和技能固化相关的模块，并参考技术报告。

#3 Shell 连续 2 天

android-reverse-engineering-skill

Claude Code skill to support Android app's reverse engineering

Stars **2,917** Forks **296** Today **+538**

https://github.com/SimoneAvogadro/android-reverse-engineering-skill

项目简介：这是一个为 Claude Code 设计的技能插件，专门用于 Android 应用逆向工程。它通过自动化流程，帮助开发者或安全研究人员在没有源代码的情况下，反编译 APK 等文件并提取出关键的 HTTP API 信息，解决了手动逆向分析效率低下的痛点。

核心功能：1. 支持多格式（APK/XAPK/JAR/AAR）反编译，并可选用 jadx 或 Fernflower/Vineflower 引擎进行对比。2. 智能提取并文档化 HTTP API，包括 Retrofit 端点、OkHttp 调用和硬编码 URL。3. 追踪从界面层到网络层的完整调用链路。4. 提供应对代码混淆（如 ProGuard）的分析策略。

适用场景：主要面向安全研究员、渗透测试人员和需要对第三方 Android 应用进行 API 分析或安全审计的开发者。典型场景包括：分析应用通信协议、进行安全漏洞挖掘、学习特定应用的架构实现，或是为没有文档的私有 API 编写客户端。

技术亮点：其亮点在于将多个成熟的命令行工具（jadx，Fernflower）整合为一个高度自动化的、可通过自然语言交互的 AI 技能。它并非重复造轮子，而是通过 Shell 脚本编排工作流，并内置了针对流行网络库（Retrofit/OkHttp）的智能模式识别，显著提升了逆向工程的分析效率和友好度。

上手建议：新用户应从阅读项目 `references/` 目录下的详细指南开始，了解工具链的安装与配置。最快体验方式是直接在 Claude Code 中使用 `/plugin install` 命令安装，并通过 `/decompile` 命令处理一个简单的 APK 文件来熟悉整个工作流程。

#4 Dart 连续 2 天

omi

AI that sees your screen, listens to your conversations and tells you what to do

Stars **10,054** Forks **1,686** Today **+824**

https://github.com/BasedHardware/omi

项目简介：Omi 是一个全平台、开源的“第二大脑”AI助手。它通过持续捕捉用户的屏幕内容和对话，进行实时转录与分析，旨在解决信息过载和记忆碎片化的问题，帮助用户高效管理数字生活。

核心功能：1) **全设备感知**：跨桌面、手机和可穿戴设备捕获屏幕与音频。2) **实时转录与摘要**：将音视频内容实时转为文字并生成摘要和待办事项。3) **情景化记忆聊天**：提供一个能记住所有历史所见所闻的AI聊天界面。4) **多模态处理**：整合语音识别（VAD、声纹分离）、LLM分析和结构化存储。

适用场景：非常适合知识工作者、远程会议频繁的团队以及追求极致效率的个人。在长时间线上会议、多任务处理、灵感记录和事后复盘等场景下，它能自动生成纪要并提炼行动点，减轻认知负担。

技术亮点：架构设计上采用**多语言混合栈**（前端用Dart/Flutter和Swift，后端用Python/Rust），实现了从嵌入式设备到云端的全链路。其**模块化后端**将语音活动检测、声纹分离、转录（集成Deepgram）和LLM推理等任务解耦，并通过Firestore和Redis高效管理海量情景化数据。

上手建议：普通用户可直接从官网下载各平台客户端体验。开发者若想贡献或深度定制，建议从 `desktop/` 目录的README开始，按照Quick Start的 `./run.sh --yolo` 命令快速构建和运行macOS演示版本，这是理解项目数据流最直接的入口。

#5 Jupyter Notebook 连续 4 天

dive-into-llms

《动手学大模型Dive into LLMs》系列编程实践教程

Stars **31,781** Forks **3,873** Today **+944**

https://github.com/Lordog/dive-into-llms

项目简介：这是一个名为《动手学大模型》的系列编程实践教程，由上海交通大学课程讲义拓展而来。它旨在通过“动手”实践的方式，为大模型初学者和研究者提供一套从入门到进阶的免费、系统化的学习资源，解决理论学习与实践脱节的问题。

核心功能：1. **核心应用实践**：涵盖微调部署、提示工程、知识编辑等大模型基础应用。2. **前沿专题探索**：深入数学推理、GUI智能体、多模态模型等前沿方向。3. **安全与对抗研究**：包含越狱攻击、模型水印、隐写术等AI安全领域的关键技术。4. **全流程国产化教程**：提供基于华为昇腾生态的《大模型开发全流程》公益教程。

适用场景：非常适合高校学生（尤其是相关课程）、希望系统入门大模型的开发者、以及需要快速上手大模型应用或安全研究的研究人员。它既可作为自学材料，也可作为课程设计的参考或实验手册。

技术亮点：项目以Jupyter Notebook为主要载体，理论与实践紧密结合，每章均提供课件、教程文档和可运行的代码脚本，形成“理论-指导-实操”的完整闭环。其内容不仅覆盖主流应用，更紧跟学术前沿（如R1推理蒸馏），并积极拥抱国产化技术栈，体现了很强的时效性和实用性。

上手建议：建议从第一章“微调与部署”或第二章“提示学习与思维链”开始，这是构建大模型认知的基础。对于贡献者，项目明确标注“PRs-Welcome”，可以从完善现有教程、修复问题或根据目录贡献新的前沿主题开始参与。

#6 Shell 连续 2 天

Claude-Code-Game-Studios

Turn Claude Code into a full game dev studio — 49 AI agents, 72 workflow skills, and a complete coordination system mirroring real studio hierarchy.

Stars **12,177** Forks **1,755** Today **+311**

https://github.com/Donchitos/Claude-Code-Game-Studios

项目简介：这是一个为 Anthropic 的 Claude Code 设计的 AI 代理框架，旨在将单个 AI 对话会话转变为一个结构化的虚拟游戏开发工作室。它解决了使用通用 AI 助手进行复杂项目（如游戏开发）时缺乏流程、分工和质量控制的问题，通过模拟真实工作室的层级和协作来提升产出质量与组织性。

核心功能： 1. **层级化 AI 代理系统**：包含 49 个具有明确职责的专属代理（如创意总监、首席程序员、关卡设计师），模仿真实工作室的三层管理结构。 2. **标准化工作流与技能**：提供 72 个“/”斜杠命令技能，覆盖从项目启动、系统设计、任务开发到完成的完整开发管线。 3. **自动化质量与合规检查**：通过 12 个钩子（Hooks）和 11 条规则（Rules），在代码提交、资产变更等环节自动执行验证并强制执行编码规范。 4. **多引擎支持**：为 Godot、Unity、Unreal Engine 5 三大主流游戏引擎提供了专门的代理和技能集，具备针对性。

适用场景：主要面向**独立游戏开发者**、**小型团队**或**原型制作爱好者**。他们希望在资源有限的情况下，利用 Claude Code 作为核心“开发伙伴”，并需要一个预定义的、系统化的协作框架来管理复杂的游戏开发过程，避免项目陷入混乱。

技术亮点：其核心并非复杂的算法，而在于**精巧的工程化设计**。它利用 Claude Code 的配置能力（代理、技能、钩子、规则），将软件工程中的“职责分离”、“质量门禁”和“流程自动化”理念，成功地映射并实现到了一个 AI 对话环境中，构建了一套可执行的虚拟团队协作范式。

上手建议：对于使用者，强烈建议从阅读 README 中的 **“Getting Started”** 部分开始，并尝试使用基础的 `/start` 和 `/help` 命令来熟悉框架。对于贡献者，可以首先研究 `.claude/` 目录下的代理、技能和规则定义文件，理解其组织方式，并从修复文档或添加对新工具链的支持等具体问题入手。

#7 TypeScript 连续 5 天

voicebox

The open-source voice synthesis studio

Stars **20,141** Forks **2,310** Today **+797**

https://github.com/jamiepine/voicebox

项目简介：Voicebox 是一个开源、本地优先的语音合成与克隆工作室。它旨在为用户提供一个功能强大且完全私密的替代方案，以解决对 ElevenLabs 等云端商业服务的依赖和隐私顾虑，让用户能在自己的电脑上完成从声音克隆到语音生成的全流程。

核心功能：其核心功能包括：1）支持从短音频克隆声音；2）集成5种不同的TTS引擎，支持23种语言；3）提供时间线编辑器，用于编排多角色对话或叙事；4）内置丰富的后期音效处理功能；5）提供REST API，便于集成到其他应用中。

适用场景：适用于内容创作者（如制作播客、有声书、视频配音）、开发者（构建需要语音交互的本地应用）以及对数据隐私有高要求的个人或企业用户。任何需要在本地进行高质量、定制化语音合成的场景都是其用武之地。

技术亮点：技术上，它采用 Tauri 框架（Rust + Web前端）而非 Electron 构建，带来了更小的体积和更高的原生性能。项目设计为“API优先”，并支持多种硬件加速后端（如CUDA、MLX、ROCm），体现了对开发者友好和高性能跨平台运行的追求。

上手建议：普通用户可直接从官网下载对应操作系统的安装包快速体验。开发者或有意贡献者，建议从查阅项目文档开始，了解其架构和API；如果想参与开发，可以关注使用 TypeScript 编写的前端界面或相关的模型集成部分。

#8 TypeScript 连续 2 天

craft-agents-oss

Stars **4,355** Forks **643** Today **+110**

https://github.com/lukilabs/craft-agents-oss

项目简介：Craft Agents 是一款开源的“智能体原生”桌面应用，旨在提供一个直观、强大的图形界面来管理和与各类AI智能体（Agent）协作。它解决了传统命令行或代码编辑器方式与智能体交互时不够直观、集成复杂、难以多任务管理的问题，让用户能以更接近自然对话和文档处理的方式驱动AI完成工作。

核心功能：1. **无配置连接**：通过自然语言指令即可连接 Slack、Linear 等外部服务或本地 MCP 服务器，自动处理 API 配置。2. **多会话与工作流管理**：提供类似收件箱的多会话界面，支持自定义状态流（如待办、进行中），便于任务追踪。3. **多模型与技能生态**：支持同时接入 Claude、GPT 等多种大模型，并能轻松导入、创建和管理可复用的“技能”。4. **强大的集成能力**：深度集成 Craft 文档工具，支持连接数据库、解析 OpenAPI 文档，并具备文件附件处理和自动化能力。

适用场景：适用于希望高效利用AI智能体进行自动化办公、数据操作或跨工具集成的开发者、产品经理及技术团队。典型场景包括：通过自然语言指令让AI自动处理 Jira 工单、分析Gmail邮件并总结、连接公司内部API生成报表，或在编写代码时动态调用各类工具。

技术亮点：项目基于 TypeScript 开发，采用 Electron 构建跨平台桌面应用。其架构显著体现了“智能体原生”思想，将复杂的配置和集成逻辑（如MCP协议、OAuth）封装，通过智能体本身的能力来简化用户操作。同时，它创新性地将 Claude Agent SDK 与 Pi SDK 并用，并支持实时、无需重启的动态技能加载（通过@提及）。

上手建议：新用户强烈建议通过官方提供的一键安装脚本快速体验桌面应用，并观看介绍视频理解其交互范式。开发者若想贡献或深度定制，可克隆源码（使用 Bun 运行时），从 `bun run electron:start` 开始运行和探索；由于其高度可定制性，直接通过应用内的智能体对话来描述你想要的功能或修改，本身可能就是最佳的入门方式。

#9 Python 连续 2 天

opensre

Build your own AI SRE agents. The open source toolkit for the AI era ✨

Stars **1,639** Forks **178** Today **+184**

https://github.com/Tracer-Cloud/opensre

项目简介：OpenSRE 是一个开源框架，旨在帮助用户构建和训练自己的 AI SRE（站点可靠性工程）智能体。它要解决的核心问题是：当生产环境发生故障时，证据（日志、指标、追踪等）分散各处，人工排查效率低下且压力巨大。该项目试图为 AI 驱动的生产事故调查与响应创建一个缺失的“强化学习环境”和基准测试平台。

核心功能： 1. **可定制 AI SRE 智能体**：提供基础框架，允许用户连接已有的 60+ 运维工具，定义自己的工作流，让 AI 代理自动执行事故调查。 2. **合成事故模拟与测试套件**：内置评分系统，通过模拟真实的生产故障（如 K8s、AWS 服务异常）来训练和评估 AI 代理的根因分析能力，甚至包含对抗性的干扰信息。 3. **端到端真实场景测试**：在真实的云环境（如 Kubernetes, EC2, Lambda）中运行测试，确保智能体在复杂分布式环境下的实用性。 4. **便捷部署与运维**：提供一键式安装脚本，并支持部署到 Railway 等平台，同时提供 CLI 工具进行远程服务状态查看和日志管理。

适用场景：主要面向 SRE 工程师、运维团队和平台工程师。他们可以在生产环境监控告警触发时，使用 AI 代理进行初步的、自动化的故障排查和证据收集，从而加快响应速度，减轻 on-call 负担。也适用于希望研究 AI 在运维领域应用的研究人员或开发者。

技术亮点： 1. **构建“强化学习环境”的愿景**：项目不仅提供工具，更旨在创建一个类似 SWE-bench 的、可量化评估 AI SRE 能力的训练与测试平台，这在当前领域颇具前瞻性。 2. **强调真实性与可评估性**：通过“合成测试”与“端到端测试”的组合，既保证了训练数据的规模和可控性，又确保了智能体面对真实云基础设施时的有效性。 3. **工程化集成与部署**：采用 LangGraph 作为智能体运行时，并严谨地处理数据库（Postgres）、缓存（Redis）等依赖，体现了生产可用的设计思路。

上手建议：对于使用者，建议从官方文档的 Quickstart 开始，使用 `opensre onboard` 和 `opensre investigate` 命令，用一个内置的测试告警（如 K8s 告警）来快速体验 AI 代理的调查流程。对于潜在贡献者，可以仔细阅读 `tests/` 目录下的合成与端到端测试案例，这是项目的核心，理解其模拟故障和评估代理的机制是深入参与的关键。

#10 Shell 连续 2 天

superpowers

An agentic skills framework & software development methodology that works.

Stars **158,293** Forks **13,776** Today **+1,713**

https://github.com/obra/superpowers

项目简介：Superpowers 是一个面向 AI 编码智能体（如 Claude、Cursor）的“智能体技能框架”和软件开发方法论。它旨在解决当前 AI 助手在复杂软件开发中容易迷失方向、缺乏系统化流程的问题，通过一套可组合的“技能”和强制执行的工程流程，将 AI 助手从一个简单的代码生成器，转变为能遵循严格开发规范（如 TDD、YAGNI）的“初级工程师”。

核心功能：1. **头脑风暴与设计确认**：在写代码前，先通过对话澄清需求，并以可消化的小块呈现设计供用户确认。2. **子智能体驱动开发**：将确认后的设计分解为微小任务，为每个任务启动独立的子智能体执行，并进行两阶段审查。3. **强制测试驱动开发**：在实现过程中强制执行“红-绿-重构”的 TDD 循环，确保测试先行。4. **集成式代码审查**：在任务间自动请求代码审查，并根据问题严重性决定是否阻塞流程。

适用场景：主要面向使用 Claude Code、Cursor、GitHub Copilot 等 AI 编程助手的开发者。它特别适用于需要 AI 助手独立、系统性地完成一个完整功能模块或小型项目的场景，能有效提升 AI 协作的产出质量和可预测性，减少开发者的微观管理负担。

技术亮点：其核心并非复杂算法，而在于**流程编排与约束设计**。它将经典的软件工程实践（TDD、代码审查、Git 工作流）封装成一系列在特定时机自动触发的“技能”，通过强制性的流程来引导和约束 AI 的行为模式，这本质上是一种针对大语言模型的“提示工程”与“工具使用”的系统化框架。

上手建议：对于使用者，最简单的方式是直接通过 Claude Code 的官方插件市场或 Cursor 的插件市场搜索 “superpowers” 进行安装，然后在一个新项目中尝试其完整工作流。对于贡献者，由于项目核心是 Shell 脚本编写的技能定义和流程控制，可以从阅读 `skills/` 目录下的具体技能实现开始，理解其触发条件和执行逻辑。

数据来源：github.com/trending

报告日期：2026-04-18

由 DeepSeek AI 提供智能分析 | 自动生成